

ECE 684: Natural Language Processing

Assignment 5 – Gradient Descent

Nruta Choudhari

Q: Explain what this neural network does. What are the inputs and outputs of the learned function?

A: This neural network is a unigram language model that predicts the probability distribution over a vocabulary of characters from a given input text.

The inputs and outputs of the learned function are as follows:

Inputs:

1. Vocabulary: A list of characters forms the model's dictionary
2. Training data: The input to the model is a sequence of one-hot encoded vectors, where each vector represents a character from the vocabulary. Each encoding indicates the presence of a particular character in the text.

Outputs:

1. Log probability of the characters ($\log p$): The output of the learned function is a probability distribution in the log form over the vocabulary. Specifically, for each input one-hot encoding, the model predicts the log probability of each character from the vocabulary.

This neural network uses gradient descent to optimize the parameters such that the log probability of the observed token is maximized. This is achieved by minimizing the negative log likelihood, also known as the loss, which is the core objective of the training process. The network learns to predict the distribution over characters based on the training document by iteratively updating its parameters through gradient descent.

Q. Choose a num_iterations and a learning_rate and creating visualizations of

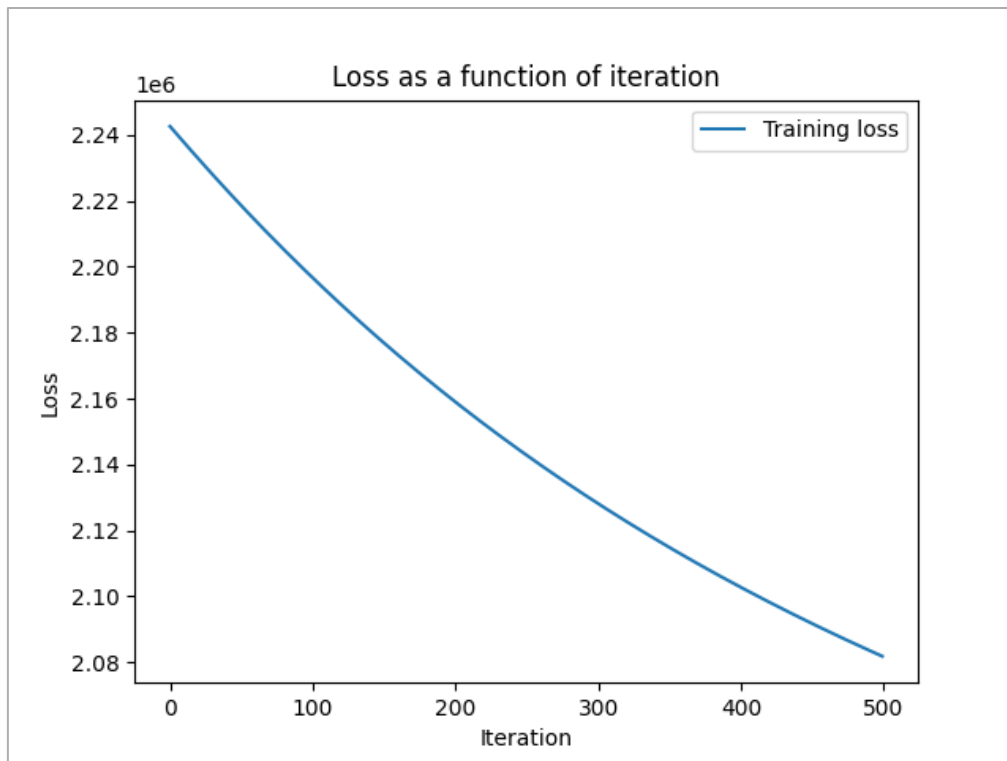
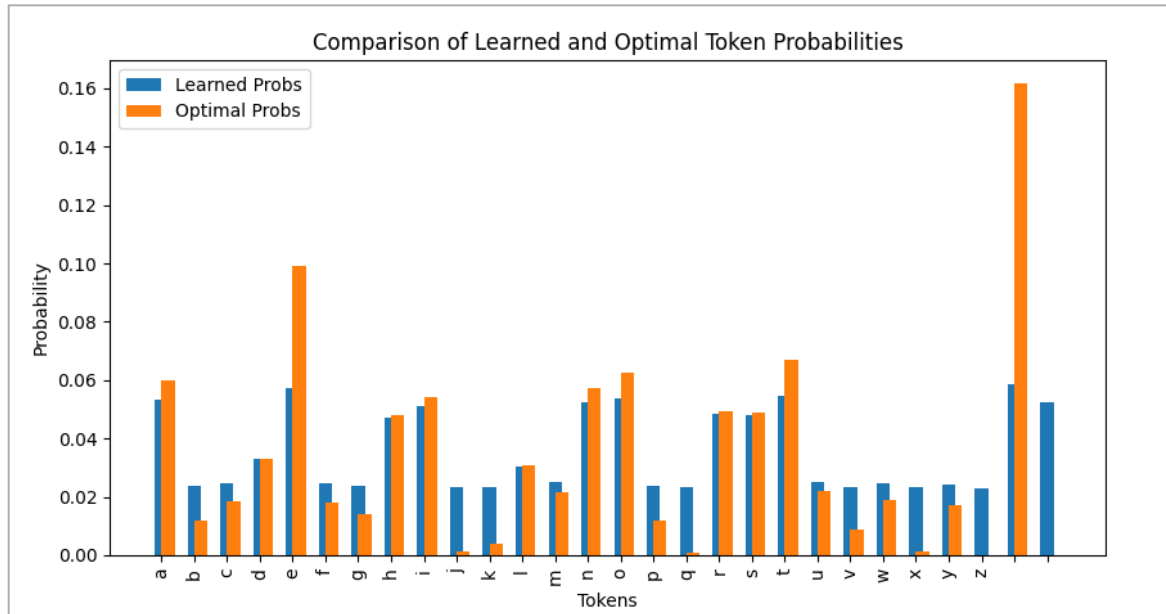
1. The final token probabilities – compare this with the optimal probabilities
2. the loss as a function of time/iteration – also include the minimum possible loss

Tweaking num_iterations and learning_rate to get reasonable good results quickly

A. Case 1: num_iterations = 500, learning_rate = 0.001

Time taken: 1.01 seconds

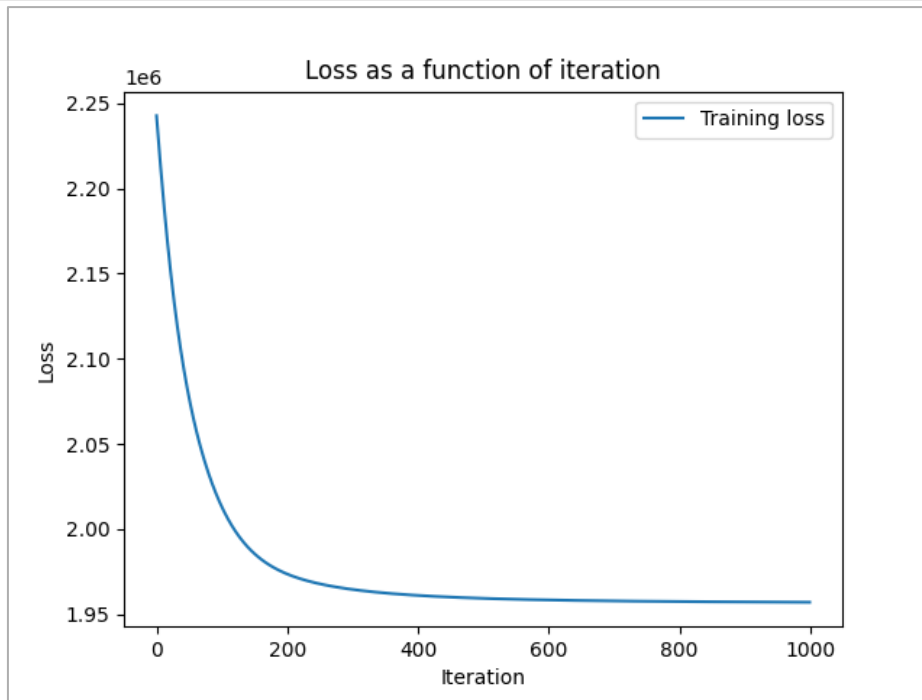
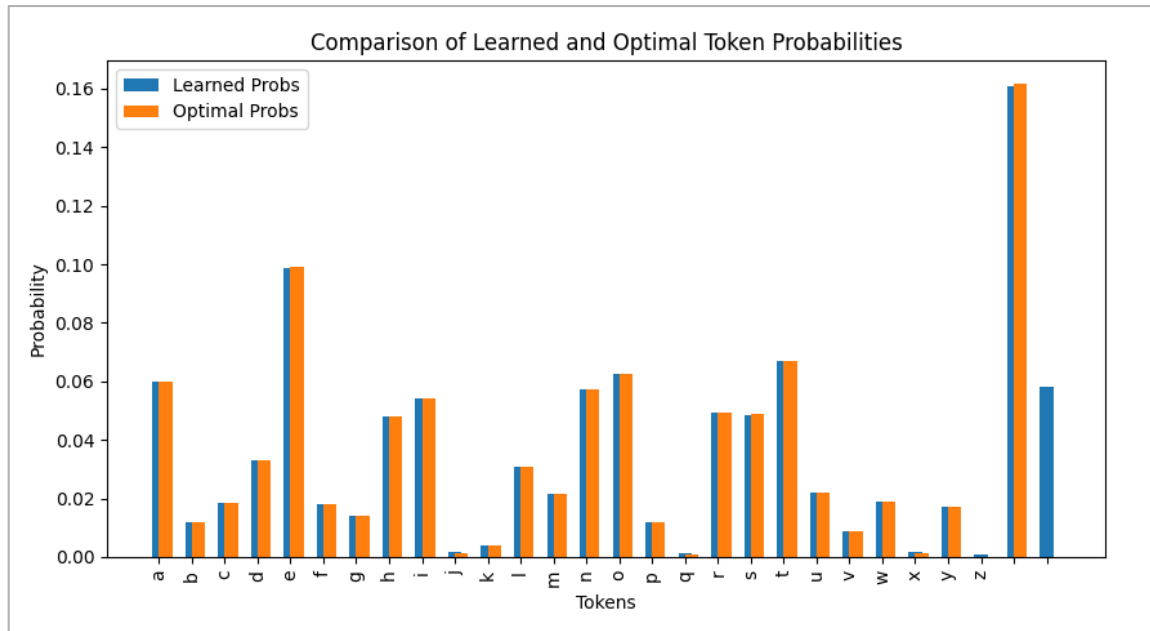
Final loss: 2081818.19



Case 2: num_iterations = 1000, learning_rate = 0.01

Time taken: 1.9 seconds

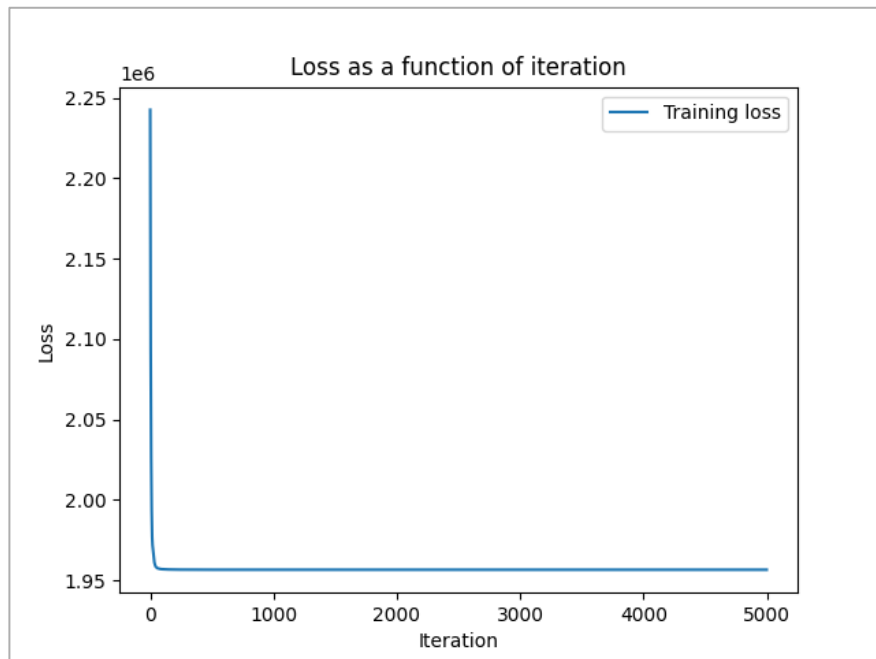
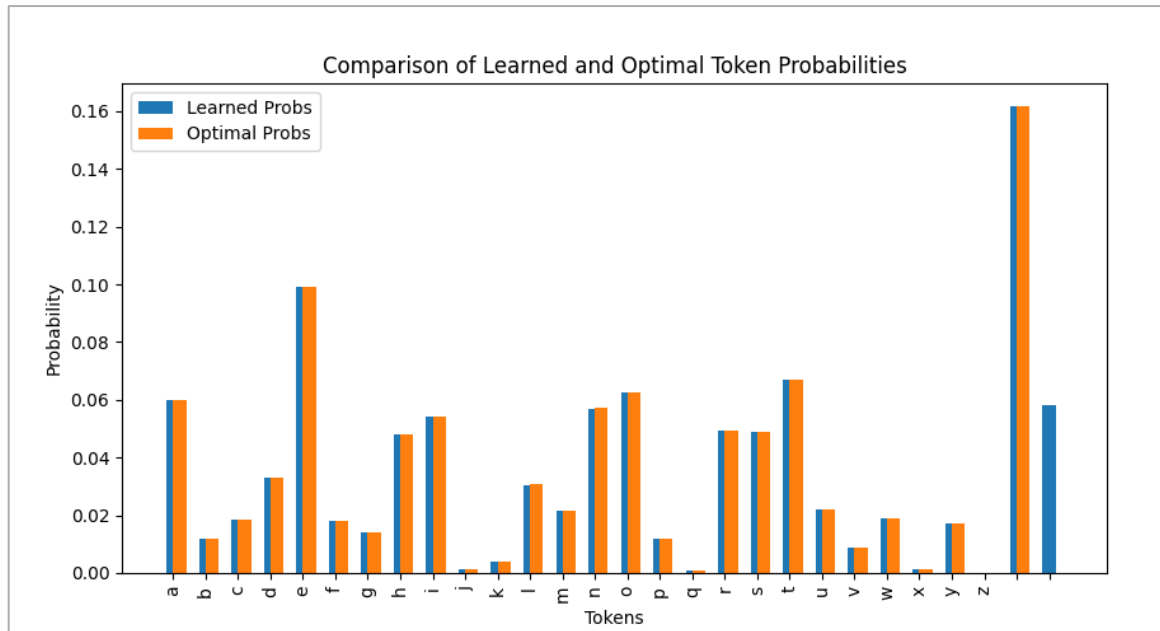
Final loss: 1957039.17



Case 3: num_iterations = 5000, learning_rate = 0.1

Time taken: 9.88 seconds

Final loss: 1956526.55



Learning rate	Num_iterations	Training time (in seconds)	Final loss
0.001	500	1.01	2081818.1863736254
0.01	1000	1.9	1957039.167347783
0.1	5000	9.88	1956526.5485818132

Thus, we can see that having a learning rate of 0.01 and the number of iterations as 1000 is optimal in terms of reducing the loss and reducing the training time.

Q. Describe how you could modify/augment this code to perform document classification?

A. The code could be modified or augmented to perform document classification in the following ways:

- Words can be used as tokens instead of characters. The vocabulary can be created out of unique words instead of individual characters.
- Replace the one-hot encoding with word count frequencies for simpler representation. This would entail using vectors which contain the frequency of words as they occur in documents/sentences.
- Introducing labeled training data where each document has a corresponding class label, for e.g., sports, politics).
- Instead of using single words (unigrams), using N-grams as tokens. This captures context and phrases, making it easier to identify themes or sentiments in the text.